SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER, CONTROL AND MANAGEMENT ENGINEERING

Neural Model Predictive Control on a Unicycle

INTELLIGENT CONTROL

Professors:
Alessandro Giuseppi

Students:
Flavio Maiorana
Flavio Volpi

Academic Year 2023/2024

Contents

Todo list

1 Introduction

outline of the work

unicycle approach

Model Predictive Control (MPC) has emerged as a powerful control strategy in robotics due to its ability to handle constraints and optimize system behavior over a prediction horizon. When combined with neural networks, which can learn complex system dynamics, it creates a promising framework for controlling nonlinear systems.

This report presents the implementation and analysis of a Neural Model Predictive Control (NMPC) scheme applied to a unicycle robot model. The unicycle serves as an excellent first-test case due to its nonholonomic constraints and nonlinear dynamics, making it both challenging and representative of real-world robotic systems, but at the same time its low complexity makes it ideal for testing without too many complications.

Our approach integrates deep learning techniques to approximate the unicycle's dynamics, which are then utilized within an MPC framework to generate optimal control inputs. We explore the effectiveness of this method in trajectory tracking tasks, comparing it with traditional control approaches.

The following sections detail the theoretical foundations, implementation specifics, and experimental results of our neural MPC system. We also discuss the challenges encountered and potential improvements for future work.

2 Model

The model must be well-defined to be used in NMPC, in such a way to obtain reliable predictions of system behavior in response to various control actions. The usage of a data-driven approach, such as neural networks, in the vehicle modeling phase has the advantage of not modeling all the nonlinear physical effects, which can be also very complex. In fact, neural networks have the capability to accurately model them. Neural networks are universal approximators, and they can be used both for designing the entire dynamic model of a system and approximating the disturbances and noises a system can be affected. Our purpose is to approximate the entire kinematic model of an unicycle with a neural model, due to its low complexity. In this section we will start describing briefly the kinematic model of a unicycle, and then we will describe the neural architecture used to approximate it.

2.1 Unicycle kinematic model

The kinematic model of the unicyle is a simplified representation of the movement of the vehicle in temrs of position and orientation. It assumes there will be no lateral deformations. The state space is represented by its position and

orientational

$$[x, y, \theta] \tag{1}$$

while the input actions are represented by longitudinal and angular velocities the vehicle can assume

$$[v, \omega] \tag{2}$$

So, the motion of the unicycle can be described through the following equations:

$$\begin{aligned} \dot{x}(t) &= v(t)\cos(\theta(t)) && (3)\\ \dot{y}(t) &= v(t)\sin(\theta(t)) && (4)\\ \dot{\theta}(t) &= \omega(t) && (5)\\ & && (6) \end{aligned}$$

2.2 Unicycle Neural model

Aggiungere figura MLP

2.2.1 Architecture

The neural network used to model the kinematic model of the unicycle is a simple MLP architecture, with two hidden layers of 128 neurons. The network has two input which are concatenated together, the current state of the unicycle (dimension 3) and the current applied actions on the vehicle (dimension 2), so it must be of dimension 5 and is represented by

$$X = [x, y, \theta, v, \omega] \tag{7}$$

The output of the network has dimension 3, and it represents the dynamics of the system, therefore it reproduces the differential equations which describe the motion of the unicycle:

$$Y = [\dot{x}, \dot{y}, \dot{\theta}] \tag{8}$$

2.2.2 Dataset

The dataset was generated by using the nominal model of the robot and generating 500 trajectories. Each trajectory is composed by 1000 steps, where the inputs are maintained constant for the whole episode. This choice was taken because it was noticed that the interval between consecutive time-steps is too small, therefore the changes about the state of the unicycle are small too, and the network was not able to approximate correctly the motion of the vehicle. On the contrary, having the same input for a lot of steps allows to understand in a better way the dynamics of the system subject to specific inputs.

Each sample is not just a pair state-action at step t, but it is a window of dimension k which contains all pairs state-action from time-step t till $t+k$ (less the last action a_{t+k} which is useless), so it is in the form

$$(s_t, a_t, s_{t+1}, a_{t+1}, ..., s_{t+k-1}, a_{t+k-1}, s_{t+k}) \tag{9}$$

This allows to have a larger window of evolution of the system with respect to just the motion executed in one single step.

2.2.3 Training and loss

As can be noticed in the previous section, the dataset does not contain the dynamics of the system, but just the evolution of the states. On the contrary, the forward pass of the network returns the vehicle dynamics. So, what we did is to pass the first pair (s_t, a_t) of a sample to the network; then we integrated the output in the sample time dt, and added to the previous state (which is also the input of the network). In this way we obtain the state $\bar{s}_{t+1}$. At this point we concatenate the new obtained state with the action a_{t+1} given by the sample dataset and we pass it to the network again, and repeating this process k times we get the states evolution according to the neural model.
Ended this mechanism, the loss is computed as the mean squared error between the neural and the nominal evolutions, i.e.

$$loss = \frac{1}{k}\sum_{i=t}^{t+k}(\bar{s}_i - s_i)^2 \tag{10}$$

3 Model Predictive Control

Model Predictive Control (MPC) is an optimal control technique where the actions are chosen solving an optimization problem, composed by a set of constraints and an objective function to be minimized over a finite horizon.

3.1 Cost function

Our objective is to minimize at each step the error with respect to the reference trajectory, both in traslational and orientational dimension, while also maintaining the module of the input actions as low as possible. A terminal state term is present too, where the traslational and orientational errors of the last horizon step are tried to be minimized. Thus, the cost function is composed by the following three terms:

1. **Spatial error**: at each step of the horizon, errors along x-axis, y-axis and error of orientation are summed together

$$J_{sp} = W_{e_x} e_x + W_{e_y} e_y + W_{e_\theta} e_\theta \tag{11}$$

where

$$e_x = x - x_{ref} \tag{12a}$$
$$e_y = y - y_{ref} \tag{12b}$$
$$e_\theta = \theta - \theta_{ref} \tag{12c}$$

2. **Terminal state**: to make sure the final stage is a good one, we add a penalty on errors along x-axis, y-axis and error of orientation in the terminal state

$$J_{term} = W_{e_{x_{term}}} e_{x_{term}} + W_{e_{y_{term}}} e_{y_{term}} + W_{e_{\theta_{term}}} e_{\theta_{term}} \tag{13}$$

3. **Input**: it penalizes high values of input controls

$$J_u = W_v v + W_\omega \omega \tag{14}$$

3.2 Constraints

The constraints of the NLP concerns some limitations on the state of the vehicle model and on the dynamics of the system. In detail, we have just two constraints:

1. **Initial state**: the initial state of the unicycle model must be the current measured state. Therefore, defining the vehicle state as x:

$$x_0 = x_{measured} \tag{15}$$

2. **Dynamics**: given the unicycle state at a certain time-step as x_t, an input at the same time-step as u_t and the dynamic model of the system defined as $\mathrm{f_{expl}}$, we establish that the next state is obtained by applying the input at the given state according to dynamic model:

$$x_{t+1} = \mathrm{f_{expl}}(x_t, u_t) \tag{16}$$

3.3 Implementation

The NMPC is developed using Acados, a software package that provides solvers for nonlinear optimal control. It is based on the symbolic framework CasADi, and provides a Python interface at an higher level.

DA FINIRE

4 Physical Informed Neural Network

In our implementation, we have used a Physical Informed Neural Network to predict the next state of the system, given the control input obtained from the Model Predictive Control.

losses

pipeline

image pinn

5 Results

what we have obtained

problems

pinn vs pinnc

6 Conclusions